

Desenvolvimento para Frontend

Introdução à TypeScript

Universidade Federal do Rio Grande do Norte

Classes

- Classes em TypeScript são baseadas no ES6, mas com recursos adicionais, como tipagem estática e visibilidade.

```
class Greeter {  
  greeting: string;  
  constructor(message: string) {  
    this.greeting = message;  
  }  
  greet() {  
    return "Hello, " + this.greeting;  
  }  
}  
let greeter = new Greeter("world");  
console.log(greeter.greet()); // "Hello, world"
```

Classes - Herança

- Suportada usando a palavra-chave `extends`. Permite criar uma nova classe baseada em uma classe existente.

```
class Animal {  
  move(distanceInMeters: number = 0) {  
    console.log(`Animal moved ${distanceInMeters}m.`);  
  }  
}  
  
class Dog extends Animal {  
  bark() {  
    console.log('Woof! Woof!');  
  }  
}  
  
const dog = new Dog();  
dog.bark();  
dog.move(10);  
dog.bark();
```

Classes – Modificadores de acesso

- Modificadores de acesso controlam a visibilidade das propriedades e métodos da classe. TypeScript oferece public, private e protected.
 - Public: Pode ser acessado de qualquer lugar.
 - Private: Só pode ser acessado dentro da própria classe.
 - Protected: Pode ser acessado dentro da classe e subclasses

Classes –

```
class Animal {
  public name: string;
  private age: number;
  protected weight: number;
  constructor(name: string, age: number, weight: number) {
    this.name = name;
    this.age = age;
    this.weight = weight;
  }
  public getAge(): number {
    return this.age;
  }
}

class Dog extends Animal {
  constructor(name: string, age: number, weight: number) {
    super(name, age, weight);
  }
  public getWeight(): number {
    return this.weight;
  }
}

const dog = new Dog("Rex", 5, 20);
console.log(dog.name); // Acessível
console.log(dog.getAge()); // Acessível via método público
console.log(dog.getWeight()); // Acessível via método da subclasse
// console.log(dog.age); // Erro: propriedade privada
```

Atividade 1

- Crie uma classe Pessoa com propriedades privadas nome e idade. Implemente métodos públicos para obter essas propriedades.

Atividade 1

- Crie uma classe Pessoa com propriedades privadas nome e idade. Implemente métodos públicos para obter essas propriedades.

```
class Pessoa {  
    private nome: string;  
    private idade: number;  
  
    constructor(nome: string, idade: number) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
  
    public getNome(): string {  
        return this.nome;  
    }  
  
    public getIdade(): number {  
        return this.idade;  
    }  
}  
  
const pessoa = new Pessoa('Ana', 30);  
console.log(pessoa.getNome()); // Saída: Ana  
console.log(pessoa.getIdade()); // Saída: 30
```

Atividade 2

- Crie uma classe abstrata Forma com um método abstrato calcularArea. Em seguida, crie duas classes que herdam de Forma: Circulo e Retângulo, implementando o método calcularArea para cada uma.

Atividade 2

- Crie uma classe abstrata `Forma` com um método abstrato `calcularArea`. Em seguida, crie duas classes que herdam de `Forma`: `Circulo` e `Retângulo`, implementando o método `calcularArea` para cada uma.

```
abstract class Forma {
    abstract calcularArea(): number;
}

class Circulo extends Forma {
    constructor(private raio: number) {
        super();
    }
    calcularArea(): number {
        return Math.PI * this.raio * this.raio;
    }
}

class Retangulo extends Forma {
    constructor(private largura: number, private altura: number) {
        super();
    }
    calcularArea(): number {
        return this.largura * this.altura;
    }
}

const circulo = new Circulo(5);
const retangulo = new Retangulo(10, 20);
console.log(circulo.calcularArea()); // Saída: 78.53981633974483
console.log(retangulo.calcularArea()); // Saída: 200
```

Referências

- <https://developer.mozilla.org/en-US/docs/Web/JavaScript/>
- <https://www.typescriptlang.org/docs/>
- <https://www.geeksforgeeks.org/>